4

FRAUD DETECTION MLOPS

Intelligent Deployment & Recovery

A visual architecture and operator guide to the Agent 4 package, its safety model, execution lifecycle, integrations, and current limitations.

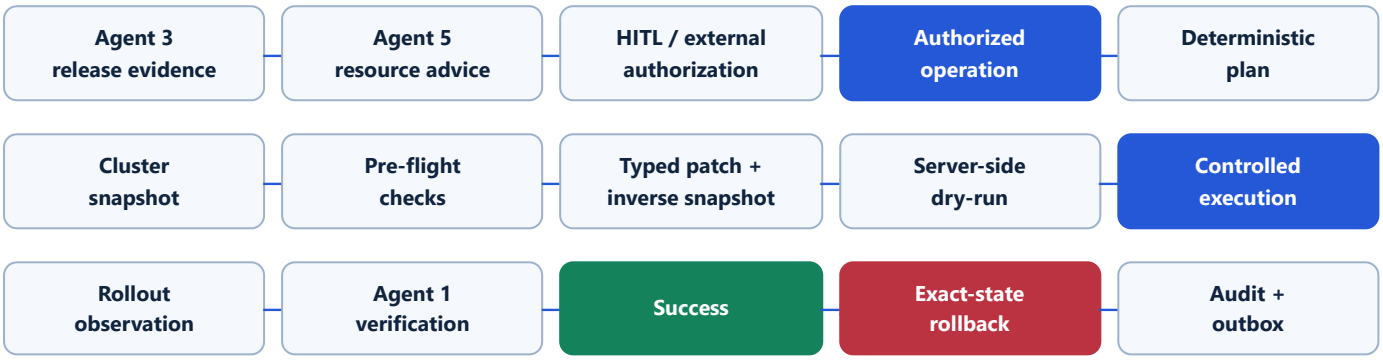
Repository: Fraud_Detection_MLOps
Package: agent4_deployment_recovery
Branch: Deployment-Agent
Guide date: 22 October 2026

ORIENTATION

How to think about Agent 4

Agent 4 is a guarded deployment state machine. It does not decide whether a release is globally acceptable. It executes an already authorized, precisely typed change and refuses to claim success until an independent verifier confirms the result.

> **The simplest mental model:** Agent 4 turns a signed intent into one exact Kubernetes patch, proves that patch is safe enough to attempt, watches the rollout, and prepares an exact inverse if recovery is required.



Responsibility boundaries

✓ Agent 4 does

- Validate typed, authorized deployment operations.
- Inspect one exact namespaced Deployment.
- Build bounded patches and exact rollback intent.
- Persist state, idempotency, locks and audit evidence.
- Observe rollout and coordinate recovery.

× Agent 4 does not

- Replace Agent 3 security policy.
- Replace Agent 5 optimization decisions.
- Invent HITL approval or an authorization issuer.
- Fabricate Agent 1 success evidence.
- Expose arbitrary Kubernetes or shell commands.

Contents

1 Architecture layers
2 Delivery phases
3 Core contracts
4 Safe execution flow
5 External adapters
6 Persistence and audit
7 Interfaces and runtime
8 Running and limitations

ARCHITECTURE MAP

The package is separated into layers

```
agent4_deployment_recovery/ ├── core/ Rules, contracts, planning and orchestration ├── adapters/
Kubernetes and cross-agent boundaries ├── persistence/ SQLite journal, locks, audit and outbox ├──
integration/ Disposable Kubernetes verification harness ├── tests/ Unit, security, recovery and
integration tests ├── cli.py Validation, inspection and dry-run interface ├── mcp_server.py Read-only
machine/agent interface ├── runtime.py Operator-controlled dependency assembly └── README.md Operational
and architectural reference
```

Layer responsibilities

Contracts	**core/schemas.py, kubernetes_models.py, execution_models.py, integration_models.py** Define precisely what data may cross a safety boundary. Unknown fields and unsupported versions fail closed.
Pure logic	**planner.py, canonical.py, preflight.py, patch_builder.py, state_machine.py** Deterministic calculations with no infrastructure mutation.
Orchestration	**coordinator.py, rollout.py, recovery.py, standalone_rollback.py** Orders the steps, handles failures and resumes safely after interruption.
External I/O	**adapters/** Explicit ports to Kubernetes, signed issuers, Agent 1, Agent 3, Agent 5 and event transports.
Durability	**persistence/sqlite_journal.py** Transactions, workload exclusion, idempotency, immutable artifacts, audit chain and outbox.
Exposure	**cli.py, mcp_server.py, runtime.py** Controls what users or other agents can reach. The assembled surface is read-only.

What was delivered by phase

Phase 0	**Contracts.** Strict Pydantic models, typed operation variants, immutable target identity, timestamps, hashes and Agent 3/5 compatibility.
Phase 1	**Safe core.** Canonical hashing, state machine, deny-by-default authorization, deterministic planning and SQLite journal.
Phase 2	**Kubernetes safety.** Inspection, capabilities, pre-flight, minimal JSON Patch and server-side dry-run.
Phase 3	**Controlled execution.** Narrow mutation adapter, rollout observation, uncertain-outcome reconciliation and exact-state recovery.
Phase 4	**Secure integration.** Ed25519 envelopes, replay protection, approval provenance, signed Agent 1 verdicts and transactional outbox.

Phase 5

Integration readiness. Read-only runtime/MCP, durable delivery leases, crash consistency, disposable-cluster harness and CI.

CORE CONTRACTS

The vocabulary Agent 4 understands

Concept	Meaning	Why it matters
`AuthorizedOperation`	An exact requested change plus target, approval, expiry, correlation and idempotency data.	Prevents free-form deployment requests.
`KubernetesTarget`	Cluster, namespace, Deployment, container, UID and expected resourceVersion.	Prevents changing the wrong object or a replaced object.
`DeploymentPlan`	Deterministic ordered steps and a plan hash.	Binds approval to one reproducible plan.
`DeploymentSnapshot`	Observed replicas, containers, probes, resources, strategy and status.	Safety decisions use real current state.
`TypedPatchPreview`	The exact guarded JSON Patch and rollback snapshot.	No unrestricted paths or last-minute patch substitution.
`ExecutionRecord`	The durable state of one execution.	Supports restart, concurrency control and audit.
`RecoveryVerification`	Agent 1's independent verdict about the deployed or restored state.	Agent 4 cannot mark success from its own observation alone.

Supported operations

Operation	Behavior	Special rule
`DEPLOY_IMAGE_DIGEST`	Changes one container image.	New image must use an immutable SHA-256 digest; current image and readiness probe must support safe rollback.
`SCALE_REPLICAS`	Changes Deployment replica count.	Observed replicas must equal the stated prior count.
`SET_CONTAINER_RESOURCES`	Changes bounded CPU/memory requests or limits.	Unspecified values are preserved; rollback remembers absent versus present fields.
`ROLLBACK_EXECUTION`	Prepares an inverse from a recorded execution.	Read-only preparation; separate authorization is mandatory and no public execution route exists.
`NO_OP`	Records an authorized decision with no infrastructure change.	Can complete without mutation or Agent 1 rollout verification.

Important hashes

Approval scope hash

Binds approval to operation, target and rollback scope.

Plan hash

Identifies the complete deterministic deployment plan.

Patch hash

Identifies the exact canonical RFC 6902 bytes accepted for execution.

Artifact hash

Protects persisted forward and rollback artifacts.

Audit hash

Links every audit event to its predecessor.

Delivery ID

Lets outbox consumers deduplicate at-least-once delivery.

SAFE EXECUTION

From request to verified result

1. **Validate the request.** Pydantic rejects unknown fields, stale timestamps, mutable deployment tags, ambiguous targets and unsupported operation types.
2. **Verify authorization.** The default verifier denies everything. A live path requires a trusted signed permit plus independently authenticated approval provenance.
3. **Create the journal record.** Idempotency and the workload lock are acquired transactionally.
4. **Inspect the Deployment.** Agent 4 reads the exact name and checks cluster identity, UID and resourceVersion.
5. **Run pre-flight.** Allowlist, expected state, probes, resources, strategy, expiry and rollback completeness are evaluated.
6. **Build the patch.** Only operation-specific paths are emitted, with server-enforced `test` operations for prior values.
7. **Dry-run the exact patch.** Kubernetes admission sees the same generated patch with `dryRun=All`.
8. **Persist before mutation.** The exact bytes and hash are durable before entering `EXECUTING`.
9. **Re-authorize immediately before mutation.** Expiry, target, plan and patch binding are checked again.
10. **Apply once.** The controlled adapter receives only a validated artifact and fresh permit.
11. **Observe rollout.** Progress is bounded and distinguishes timeout, stall, failure, cancellation and identity drift.
12. **Ask Agent 1.** Only trusted `VERIFIED` evidence can lead to `SUCCEEDED`.

State machine

```
RECEIVED → AUTHORIZED → PREFLIGHT_PASSED → PLANNED → DRY_RUN_PASSED
         → EXECUTING → OBSERVING → AWAITING_AGENT1 → SUCCEEDED
```

```
EXECUTING / OBSERVING / AWAITING_AGENT1
         → ROLLBACK_PENDING → ROLLING_BACK
         → ROLLBACK_AWAITING_AGENT1 → ROLLED_BACK
```

> **Uncertain mutation:** a timeout does not mean "nothing happened." Agent 4 re-reads the Deployment and reconciles exact fields. It never blindly retries a mutation.

> **Failed rollback verification:** if the inverse conflicts, rollout fails, scope expires or Agent 1 cannot verify, the result is `ROLLBACK_FAILED`, not a false success.

Why the patch is constrained

- Only image, replica and resource paths are generated.
- JSON Pointer tokens are escaped correctly.
- UID, resourceVersion, container name and prior field values are tested.
- Secrets, cluster-scoped objects and unrelated containers cannot be targeted.
- Callers cannot submit arbitrary JSON Patch through CLI or MCP.

FILES: CORE

What each core file represents

`schemas.py`
Primary operations, targets, plans, states, approvals and execution records.

`canonical.py`
Stable JSON normalization and SHA-256 hashing.

`authorization.py`
Authorization protocol, denial model and deny-all default.

`planner.py`
Pure conversion from authorized operation to deterministic plan.

`state_machine.py`
Complete allowlist of legal execution transitions and terminal states.

`kubernetes_models.py`
Cluster connection, Deployment snapshots, capabilities, pre-flight, patches and dry-run results.

`preflight.py`
Structured PASS/BLOCK checks against policy and observed state.

`patch_builder.py`
Minimal guarded JSON Patch plus exact inverse data.

`dry_run.py`
Explicitly opted-in server-side admission validation.

`execution_models.py`
Persisted patch artifacts, permits, mutation, rollout and reconciliation results.

`rollout.py`
Bounded observation of Deployment convergence and failure conditions.

`recovery.py`
Uncertain-outcome reconciliation and exact inverse patch generation.

`standalone_rollback.py`
Read-only rollback preparation from trusted journal evidence.

`integration_models.py`
Signed authorization, approval provenance, Agent 1 and outbox contracts.

`coordinator.py`
The central engine that sequences authorization, safety, mutation, observation and recovery.

`errors.py`
Typed failures for policy, journal, Kubernetes, signatures, replay and integration.

`__init__.py`
Marks the directory as a Python package without activating components.

The coordinator is powerful but dependency-controlled

The coordinator cannot contact a cluster on its own. It receives interfaces for authorization, inspection, dry-run, mutation, rollout observation, verification and persistence. Production safety therefore depends on both its logic and which adapters the private runtime chooses to inject.

> **Current assembly:** the public runtime does not inject the live mutation adapter. A production-capable class existing in source code is not the same thing as a reachable production mutation endpoint.

Operation-specific pre-flight

Check	Image	Scale	Resources
Exact UID/resourceVersion	Required	Required	Required
New immutable image digest	Required	Not relevant	Not relevant
Current immutable image	Required for safe inverse	Not required	Not required
Readiness probe	Required	Policy/rollout dependent	Rollout affecting
Expected prior replicas	Not relevant	Required	Not relevant

Check	Image	Scale	Resources
Request ≤ limit	Not relevant	Not relevant	Required

FILES: ADAPTERS & PERSISTENCE

How Agent 4 reaches the outside world

agent1_verification.py

Defines the independent verification port. When no trusted implementation exists, it returns `UNAVAILABLE`.

compatibility.py

Reads actual Agent 3 and Agent 5 schemas. Their outputs remain evidence/advice and never become authorization.

kubernetes.py

Uses the official client for exact Deployment inspection and `dryRun=All`. There is no silent context fallback.

kubernetes_mutation.py

Contains a disabled default and a tightly allowlisted controlled adapter. It is not wired into public interfaces.

signed_contracts.py

Checks Ed25519 signatures, issuer, audience, expiry, scope, replay and approval provenance.

outbox.py

Provides transport-neutral at-least-once dispatch with leases, bounded retries and acknowledgements.

SQLite execution journal

`persistence/sqlite_journal.py` is the durable safety backbone:

Idempotency
A key cannot be reused for a different plan.

Workload lock
Only one active execution may own a workload.

Optimistic state
Transitions reject stale record versions.

Artifacts
Patch content is immutable and hash checked.

Audit chain
Every event hashes the previous event.

Outbox
Events and state commit in one transaction.

> **Audit limitation:** hash linking makes tampering detectable, but SQLite is not an independently administered audit service. Production needs protected storage, backups, access controls and external retention.

External contracts still needed

Provider	What Agent 4 expects	Current status
Orchestrator	Typed operation plus signed, hash-bound authorization permit.	Contract exists; provider not implemented here.
HITL service	Authenticated provenance for the approval reference.	Deny-all default; no fabricated approval.
Agent 1	Signed, one-time verdict bound to execution and expected state.	Interface exists; default is unavailable.
Agent 3 CI	Assessment plus trusted provenance binding digest to build.	Schema adapter exists; assessment alone is insufficient.
Agent 5	Recommendation plus exact workload resolution.	Translator produces only a proposal.

Provider	What Agent 4 expects	Current status
Event transport	Delivery of outbox events with consumer deduplication.	Dispatcher exists; no Kafka/gRPC transport.

INTERFACES

What can be run today

CLI surface

Command	Purpose	Cluster contact
`validate-operation`	Validate and summarize an operation.	No
`plan`	Create a deterministic deployment plan.	No
`validate-plan`	Validate an existing plan and hash.	No
`validate-agent3`	Read Agent 3 evidence without authorizing it.	No
`validate-agent5`	Read Agent 5 advice without authorizing it.	No
`inspect --snapshot`	Display a stored Deployment snapshot.	No
`inspect --contact-cluster`	Read one explicitly configured Deployment.	Explicit opt-in
`preflight`	Produce machine-readable PASS/BLOCK reasons.	No
`patch-preview`	Display the exact guarded patch and rollback snapshot.	No
`server-dry-run`	Ask Kubernetes admission to validate the patch without persistence.	Explicit opt-in

MCP surface

The default MCP server exposes only capability discovery, operation validation, plan preview and Agent 3/5 evidence checks. Operator assembly may add constrained inspection and authenticated, redacted journal reads.

> **Never exposed through MCP:** deploy, execute, live apply, rollback execution, shell, kubectl, caller-provided trust keys, caller-provided kubeconfig or arbitrary patch input.

Runtime assembly

`runtime.py` reads one operator-owned absolute configuration path. External callers cannot inject keys, clients, allowlists or permits. It validates `READ_ONLY` and future configuration shapes but always reports live execution unavailable.

Disposable Kubernetes integration harness

The `integration/kubernetes/` folder contains:

- `harness.py` — strict opt-in and exact test-target configuration.
- `rbac.yaml` — a namespaced test ServiceAccount/Role/RoleBinding limited to named Deployment get/patch.
- `README.md` — prerequisites and cleanup responsibility.

The optional tests verify exact reads, dry-run non-persistence, stale version rejection, RBAC denial and bounded rollout observation. They must target a disposable cluster.

RUNBOOK

How to explore Agent 4 locally

1. Activate the isolated environment

```
.\.venv-agent4\Scripts\Activate.ps1
python --version
```

2. Run all tests

```
python -m pytest agent4_deployment_recovery\tests -v
```

3. Watch specific behavior

```
# Execution, Agent 1 verification and rollback
python -m pytest agent4_deployment_recovery\tests\test_phase3_coordinator.py -v

# Kubernetes safety checks and typed patches
python -m pytest agent4_deployment_recovery\tests\test_preflight.py -v
python -m pytest agent4_deployment_recovery\tests\test_patch_builder.py -v

# Signatures, expiry, forgery and replay protection
python -m pytest agent4_deployment_recovery\tests\test_phase4_signed_contracts.py -v

# Process crash and restart consistency
python -m pytest agent4_deployment_recovery\tests\test_phase5_crash_consistency.py -v
```

4. Validate other-agent fixtures

```
python -m agent4_deployment_recovery.cli validate-agent3 `
  agent4_deployment_recovery\tests\fixtures\agent3_release_assessment.json

python -m agent4_deployment_recovery.cli validate-agent5 `
  agent4_deployment_recovery\tests\fixtures\agent5_resource_recommendation.json
```

5. Discover the CLI and MCP server

```
python -m agent4_deployment_recovery.cli --help
python -m agent4_deployment_recovery.cli preflight --help
python -m agent4_deployment_recovery.mcp_server
```

> **Best current visualization:** the verbose tests show success, denial, timeout, uncertain apply, restart, verification failure and rollback paths. CLI commands render contracts, plans, reasons and patches as formatted JSON.

Continuous integration

`.github/workflows/agent4-quality.yml` installs Python 3.12 dependencies, runs pytest, Ruff and `pip check`, with read-only repository permission and no deployment permission.

> **Current trigger detail:** the pull-request trigger targets the `Deployment-Agent` branch. A PR from `Deployment-Agent` into `main` does not match that Agent 4 workflow trigger; Agent 3's release gate is the gate evaluating PR #3.

REALITY CHECK

Executable, but not yet a deployment product

Implemented and runnable

- Strict operation and integration contracts.
- Planning, hashing and state transitions.
- SQLite journal, locks, audit and outbox.
- Kubernetes inspection and server dry-run.
- Constrained controlled-mutation component.
- Rollout observation and exact recovery logic.
- Signed-contract validation and replay defense.
- Read-only CLI/MCP and extensive tests.

Not yet integrated

- No production Orchestrator authorization issuer.
- No trusted external HITL provenance service.
- No production Agent 1 verifier transport.
- No Kafka or gRPC outbox transport.
- No operator-reviewed live composition root.
- No production cluster/RBAC/network validation.
- No web dashboard or deployment timeline UI.
- No canary, blue/green or weighted routing.

Safety invariants to remember

1. Agent 3 evidence and Agent 5 recommendations never authorize mutation.
2. The default authorization and mutation implementations deny live actions.
3. The exact dry-run patch bytes must be the exact persisted execution bytes.
4. UID, resourceVersion and prior field values protect against race conditions.
5. SQLite constraints—not only process locks—enforce idempotency and workload exclusion.
6. Rollout convergence never substitutes for independent Agent 1 verification.
7. Unknown schemas, strategies, states or ambiguous responses fail closed.
8. A timeout causes reconciliation, not a blind retry.

> **Bottom line:** Agent 4 is not static documentation. It is an executable, heavily tested backend control plane with read-only operational interfaces. It is intentionally not yet exposed as a live deployment service or visual dashboard.

Short glossary

Term	Plain meaning
Fail closed	If required evidence is missing or ambiguous, block instead of assuming success.
Idempotency	Repeating the same request does not create a second, different execution.
resourceVersion	Kubernetes concurrency token proving the object has not changed since inspection.
Server dry-run	Kubernetes validates admission without persisting the request.
Exact-state rollback	Restore only the exact prior value that Agent 4 changed.
Transactional outbox	State and an event are stored atomically, then the event is delivered later.
HITL	Human in the loop; approval provenance must come from an external trusted source.

Generated from the implementation on the Deployment-Agent branch. This guide explains the repository state; it does not activate live mutation or contact Kubernetes.